

HUMANS.md — Working with AI on CollectionBuilder-CSV

This guide is for *you*, the human. Its companion, **AGENTS.md**, is written for the AI coding assistant. Coding agents (Cursor, Copilot, Codex, and others) read **AGENTS.md** automatically at the start of a session, so the framework's architecture rules already reach the AI without you pasting them into every prompt. This file covers what the AI can't do for you: setting intent, prompting well, and reviewing what it produces.

What the two files do

- AGENTS.md** teaches the agent the framework's rules — the customization hierarchy, what never to edit, the feature includes. It is the single source of truth.
- HUMANS.md** (this file) teaches you how to direct and check an agent that already knows those rules.

If a rule about the architecture ever needs updating, change it in **AGENTS.md**. Treat that file as authoritative so the files don't drift apart. (Claude Code and Cowork read **CLAUDE.md**, which the repo ships as a one-line pointer importing **AGENTS.md** — so they get the same rules with no setup on your part.)

The mental model

CollectionBuilder-CSV is **data-driven**: almost every change happens in a CSV or YAML file, not in HTML or Liquid. When working with AI, your job is mostly to:

- Say what you want in terms of **outcomes** ("add a subject facet to browse"), not file edits.
- Watch the diff for the handful of moves that signal the AI ignored the architecture.
- Rebuild and check **one change at a time**.

The agent knows the rules. You own the intent and the review.

Prompting: outcomes, not implementations

The agent already knows which file controls what, so describe the result you want and let it route the change to the right layer.

Instead of	Try
"Edit browse.html to show subjects"	"Add a subject facet to the browse page"
"Make the navbar navy"	"Change the navbar background to navy"
"Fix the navigation"	"The About link is missing from the navbar — add it"
"Add an image to the about page"	"Add a featured image of demo_001 near the top of the about page"

Name objectids, field names, and pages when you know them. Vague verbs like "show," "fix," and "update" force the agent to guess.

Reviewing the diff: red flags

Because the architecture has only a few hard rules, you can review most AI changes by scanning **which files were touched**. For routine work, any of these usually means the agent took a shortcut it shouldn't have — stop and ask it to redo the change at the config level:

- **Edited anything in `_layouts/` or `_includes/`** (especially `collection-nav.html`) — a config CSV almost always does the job instead.
- **Created a file under `items/`** — item pages are auto-generated from the metadata CSV; manual ones break the build.
- **Edited `_base.scss`, `_pages.scss`, or `_theme-colors.scss`** — custom styles belong only in `_sass/_custom.scss`.
- **Added `.csv` to the metadata: pointer** in `_config.yml` — it should never carry the extension.
- **Wrote `site.data.metadata`** — must be `site.data[site.metadata]` (bracket notation).
- **Used Bootstrap 4 classes** (`ml-/mr-`, `data-toggle=`) — this is Bootstrap 5.
- **Prefixed a `display_template` value** with `item/` — use `image`, not `item/image`.

A clean change, by contrast, touches `_config.yml`, a `_data/*.csv`, `_data/theme.yml`, a page in `pages/`, or `_sass/_custom.scss` — and nothing else. (Deliberate larger changes are the exception — see "Going bigger" below.)

When to slow the agent down

Let the AI move freely on additive, reversible changes (a new nav link, a facet, a feature include on a page). Ask it to explain its plan **before** editing when a change is broad or hard to undo:

- Restructuring or reordering columns in `_data/<metadata>.csv`
- Bulk edits across many item rows
- Changing `display_template` for a whole collection
- Anything that rewrites a config CSV's structure rather than adding or removing a row

Going bigger: new templates, pages, and redesigns

The red-flags list above assumes routine, additive work. Larger changes — a new item display template, a brand-new kind of page, a full visual redesign — legitimately leave the config layer and touch `_layouts/`, `_includes/`, or `_sass/`. That's fine. The difference is that you're directing it **deliberately**, with a clear target in mind, rather than catching the AI taking a shortcut. When you go big, your job shifts from "watch which files it touches" to "give it a sharp brief and review the result against your vision."

Briefing the AI for a big change

The single most useful thing you can do is **point the AI at an existing example to mirror**. CB-CSV is full of working patterns; an AI that copies the structure of an existing display template or page will stay consistent with the framework far better than one inventing from scratch. A strong brief usually includes:

- **The outcome, described richly** — what it should do, look like, or feel like. Adjectives, a reference site you admire, a sketch, even a screenshot all help.

- **An existing file or feature to base it on** — "model it on the existing `image` display template," "match the structure of `browse.html`."
- **The rules that still apply** — feature includes for content, custom CSS in `_sass/_custom.scss` only, Bootstrap 5, and registering new `display_template` values so they resolve.
- **A request to plan first** — for anything non-trivial, ask the AI to explain its approach before it edits, so you can course-correct cheaply.
- **Incremental delivery** — one piece at a time, building and checking as you go.

You don't need to know which files to edit. The AI does, from `AGENTS.md`. You bring the vision and the judgment.

Sample prompts

A new display template (e.g., an `event` layout, a IIIF image viewer, or a 3D object viewer):

I want a new item display template called `event` for items that represent an event with several associated objects (photos, a program, documents). Base it on the existing `compound_object` display template — keep the same underlying mechanism for pulling in child objects via `parentid` — but give it a new design: the event's date and location featured in a header at the top, the description below that, and the associated objects arranged as a gallery beneath. Make sure `display_template: event` resolves to it, use the existing feature includes where they fit, keep the markup Bootstrap 5, and don't touch the other templates. Walk me through your plan before you edit anything.

A new page (beyond a standard content page — a custom landing page, a narrative, a visualization):

Create a new page at `pages/timeline-story.md` that tells the collection's history as a scrolling narrative: short text sections interleaved with full-width images and a couple of galleries filtered by decade. Build it from the existing feature includes rather than raw HTML wherever possible; if you need anything custom, put styles in `_sass/_custom.scss` and explain what you added. Use these objectids for the images: [list]. Show me an outline of the page structure before writing it.

Overall look and feel (a visual redesign):

I want to redesign the site's look to feel calmer and more editorial — generous whitespace, a serif display font for headings, a muted earth-tone palette, and understated navigation. Here's a site whose feel I like: [URL]. Start by proposing the changes you'd make in `_data/theme.yml` and `_data/config-theme-colors.csv` — fonts, colors, navbar — since those are the right first layer. Only reach for `_sass/_custom.scss` for what those files can't express, and tell me each time you do. Give me the `theme.yml` and color changes first so I can see the foundation before we refine spacing and details.

Notice the shape these share: a clear target, an existing pattern or reference to anchor to, the architecture rules restated, and a "plan first" or "foundation first" gate so you stay in control. Big ambitions are welcome here — the framework rewards them when you brief them well.

Working rhythm

1. **One change at a time.** Rebuild (`bundle exec jekyll serve`) and look before moving on.
2. **Eyeball the CSV row** the agent added — small typos break features quietly.

3. **Keep `AGENTS.md` current** as the framework evolves; every agent inherits the improvements you make there.
4. **Your chat instructions override `AGENTS.md`.** If you tell an agent to do something the file forbids, it may comply — so don't fight your own rules without a reason.

Where to find documentation

Point the agent at the `docs/` folder for details it needs — `docs/metadata.md`, `docs/markup.md`, `docs/maps.md`, `docs/color_theme.md`, `docs/advanced_theme.md`, and the rest. The official docs live at <https://collectionbuilder.github.io/cb-docs/>, and framework questions belong on the [discussion forum](#).